IP Routing

Fundamentals and Advanced Applications

Study Notes: RIB/FIB, Route Types, and Multi-Protocol Route Import

Contents

1 Overview of IP Routing

When a router receives an IP packet, it matches the packet's **destination address** against its routing table:

- **Match found** → the router forwards the packet according to the outbound interface / next hop in that entry.
- **No match found** → the router has no information to guide forwarding, so it **discards** the packet.

2 RIB and FIB

Every routing-capable device maintains two key tables.

2.1 RIB – Routing Information Base

- Located on the **control plane**.
- Despite being called the "routing table," the RIB does *not* directly guide forwarding.
- The router downloads its **optimal routes** from the RIB into the FIB. Whenever RIB entries change, the FIB is synchronized immediately.
- Because the two stay consistent, and the RIB is easier to read, people commonly describe forwarding using the RIB – but technically the router queries the **FIB**, not the RIB.

2.2 FIB – Forwarding Information Base

- Located on the **data plane** – also called the *forwarding table*.
- Each entry specifies the **outbound interface** and **next-hop IP address** for reaching a destination.
- This is the table actually consulted when forwarding real packets.

2.3 RIB structure: two sub-tables

- **Protocol routing table** – one per protocol (e.g. OSPF), storing everything that protocol learned. If a protocol has multiple paths to the same destination, it picks its own best one internally (*active*), keeping the rest as *inactive* backups.
- **Local core routing table** – still on the control plane. For each destination, the router compares the *active* candidates offered by each protocol and keeps only the single best one (by Preference, then Cost). This is what gets pushed down to the FIB.

Two levels of competition:

1. *Within* one protocol (e.g. two OSPF paths to the same destination) → decided by **Cost**.
2. *Across* protocols (e.g. OSPF vs. Static to the same destination) → decided by **Preference** first, **Cost** only as a tiebreaker.

2.4 Where RIB and FIB live on a chassis

- **RIB**: centralized – exactly **one** RIB, computed once on the main control board.
- **FIB**: distributed – the computed routes are pushed down to **every slot/line card**, so each card can forward packets locally at wire speed without asking the control board per packet.

Command to inspect the FIB: `display fib [slot-id]` – `slot-id` lets you check the FIB copy on a specific card in a modular chassis.

3 Key Fields in a Routing Table

Field	Description
Destination	Destination IP address or network segment
Mask	Subnet mask paired with the destination address
Proto	Protocol through which the route was learned (static, OSPF, IS-IS, ...)
Pre (Preference)	Trust level of the protocol; compares routes from *different* protocols
Cost	Metric of the route; compares routes from the *same* protocol
NextHop	Next-hop device address
Interface	Local outbound interface used to forward packets

4 FIB Table Flags

Real flags documented by Huawei for `display fib` output:

Flag	Meaning
G	Gateway route – next hop is another router
H	Host route – next hop is the destination host itself
U	Up – route is active/usable
S	Static route
D	Dynamic route (learned via OSPF, IS-IS, BGP, RIP, ...)
B	Black hole route – outbound interface is Null0, packet is silently dropped
L	Vlink / ARP-or-IS-IS-generated route (platform dependent)

Note: G and H are mutually exclusive (a next hop is either another router or the destination itself), so they never appear together. Real combinations look like `HU`, `DGU`, `SU` – not all letters stacked together.

Other FIB fields: **Total number of Routes**, **Timestamp** (entry lifetime in seconds), **TunnelID** (nonzero ⇒ forwarded through a tunnel, e.g. MPLS; 0 ⇒ no tunnel).

5 Longest Match Principle

When looking up the FIB, the router performs a bitwise **AND** between the packet's destination IP and each entry's mask, then compares the result against that entry's network address. **Multiple entries can match** – the router always selects the one with the **longest (most specific) mask**.

Example: destination `10.3.3.3`, with FIB entries `10.0.0.0/8`, `10.3.0.0/16`, `10.3.3.0/24` – all three match, but `/24` wins because it is the most specific.

6 Route Types

- **Direct route** – automatically generated for the subnet of a directly connected interface.
- **Static route** – manually configured by an administrator.
- **Dynamic route** – learned via a dynamic routing protocol (OSPF, IS-IS, BGP).

7 Dynamic Routing Protocols

7.1 Classification by scope

- **IGP (Interior Gateway Protocol)** – runs *inside* one Autonomous System (AS). Examples: **OSPF**, **IS-IS**. Both use the **Shortest Path First (SPF)** algorithm based on link-state information.
- **EGP (Exterior Gateway Protocol)** – runs *between* ASs. Example: **BGP**, a path/distance-vector protocol used for inter-AS routing (the protocol underlying the global Internet backbone).

Autonomous System (AS): a group of IP networks under one administrative entity (typically an ISP), sharing the same routing policy, identified by a unique AS number.

7.2 Routing processes

- A router can run **multiple OSPF/IS-IS processes**, independent of one another, assigned per service type.
- An OSPF **process ID** has *local* significance only – it does not need to match between neighboring routers; packets can still be exchanged between routers running different process IDs.

8 Route Recursion

A route can only forward traffic when it has a **directly connected next hop**. However, **static** and **BGP** routes are often configured with an **indirect** next hop. **Route recursion** is the process of repeatedly searching the routing table for that next hop, hop after hop, until a directly connected next hop is found.

This lets admins point routes at a stable logical address (e.g. a loopback), rather than a fragile directly-connected interface IP – especially common with BGP, whose next hop is often reachable only via an underlying IGP route.

9 Data Forwarding Process

Consider a packet traveling from a source host, through a chain of routers, to a destination host on a remote network:

1. **The source host sends the packet to its default gateway.** A host that is not on the same subnet as the destination cannot deliver the packet directly, so it forwards it to its configured gateway.
2. **Each intermediate router forwards the packet, hop by hop.** At every router along the path, the packet's destination IP is looked up in that router's own routing table (FIB) to determine the next hop and outbound interface. The packet is then forwarded accordingly.
3. **The final router delivers the packet locally.** When a router finds that the destination IP belongs to the network segment of one of its own directly connected interfaces, it delivers the packet locally rather than forwarding it to another router.

Hop-by-hop forwarding: each router decides only its own next hop, based only on its own routing table – no router needs to know the full end-to-end path.

10 Advanced Application: Multi-Protocol Interworking

10.1 Why This Comes Up

A common real-world case: two previously independent networks are merged (for example, after two companies merge). One network runs **OSPF**, the other runs **IS-IS**. After the merger, the two networks must be **integrated** so that service traffic can be exchanged normally across the whole combined network – the requirement is networking based on **routing**.

Core problem: OSPF and IS-IS are different dynamic routing protocols – they **cannot directly exchange routing information** with each other.

Solution sketch: a routing protocol can be deployed on the network segments that connect the two domains at their border. Certain edge devices at that border are configured to run **both** protocols simultaneously, acting as **border devices** that bridge the two routing domains.

10.2 General Principle

On a **large-scale enterprise network**, a single routing protocol usually cannot meet all requirements – in most cases **multiple routing protocols coexist**. Different protocols are deployed in different parts of the topology based on **service logic** or **administrative management**, keeping the routing hierarchy **clear and controllable**. In such an environment, **interworking based on routing** must still be implemented across domain boundaries.

11 Basic Concept of Route Import

11.1 Two ways to achieve routing-based interworking

1. **Replan and modify network-wide routing protocols** – unify everything under one protocol. *Complex.*
2. **Configure only the edge devices (Route Import)** – transmit routing information between OSPF and IS-IS at the border. Does **not** change the original topology and is **easy to deploy**, but **loops may occur** if not carefully controlled.

11.2 Definition

Route import is the process of **advertising routing information from one routing protocol into another**. It allows routing information to be transmitted between different routing protocols, and **route control** can be applied during import to flexibly control service traffic.

11.3 Route import is directional

If routes are imported **from Protocol A into Protocol B**:

- Protocol B **learns** A's routes.
- Protocol A does **NOT** learn B's routes – unless import from B to A is *also* configured.

Consequence if only A → B is configured: devices in domain B *can* reach devices in domain A (B has the routes). Devices in domain A *cannot* reach devices in domain B (A never learned B's routes, so there is no matching entry and the packet is discarded).

11.4 Key points during route import

- **Route preference**
- **Route injection**
- **Route metric**
- **Route convergence time**

12 Route Import: Route Preference

12.1 The problem it can cause

When the same destination network is reachable via two different protocols after mutual route import – one path short and direct through the originating protocol, the other a much longer path that loops through several routers and a second import point – the router chooses between them purely based on **preference**, not physical path length. Since a **lower** preference value wins, a protocol with a lower default preference (e.g. IS-IS) can end up being chosen over a shorter path learned through a protocol with a higher preference (e.g. an OSPF external route), even though that path is objectively longer and less efficient. This is a classic **suboptimal routing** problem caused by relying on default preference values without adjustment.

12.2 Huawei default route preferences

Route Source	Preference
Direct	0
OSPF (internal)	10
IS-IS	15
Static	60
OSPF ASE (external)	150
OSPF NSSA	150
IBGP	255
EBGP	255

Lower number = more preferred. These defaults are vendor-specific (Huawei); other vendors use different values/conventions.

13 Route Import: Route Injection

13.1 Concept

Route injection means controlling exactly **which** routes get imported from one protocol into another, rather than importing everything blindly. It is typically controlled using route-policies, ACLs/prefix-lists, and **route tags** – tagging a route at the point it is first imported, so that later import points can recognize and reject a route that would otherwise be re-imported back into the domain it originally came from.

13.2 The redistribution loop problem

When route import is configured at more than one border point between two protocol domains, a route that starts in one domain can be imported into the second domain, and then – if not filtered – imported **back** into the first domain at a different border device. The route now exists in its original domain via two paths: the original, correct one, and a re-injected one that takes a long detour through the other domain and back. This duplicate, longer path is exactly the kind of **routing loop risk** associated with edge-device-only route import, and it is why **route injection (filtering)** at each border device is essential whenever import is configured in more than one place.

14 Route Import: Route Metric

Each protocol calculates its metric differently (OSPF cost from bandwidth, IS-IS metric, RIP hop count, BGP path attributes), so metrics **cannot be directly carried over** between protocols. On import, the receiving protocol assigns a **default metric** to the route – if left unadjusted, this default may not reflect the real path cost, contributing to the same kind of **suboptimal routing** seen in the preference case. Administrators can manually set the metric during import configuration.

14.1 The three levers of route import, together

Lever	Controls
Preference	Which *protocol's* route wins, when multiple protocols offer a route to the same destination
Metric	Which *specific route* wins, among routes of the *same* protocol to the same destination
Injection (filtering)	*Which* routes get imported at all – preventing unnecessary routes or loops

15 Route Import Commands (OSPF)

15.1 Basic command

```
import route { bgp | direct | static | isis <process-id> } [ process
                                ospf-process-id ]
```

Only the **source** keyword changes across scenarios (`bgp`, `direct`, `static`, `isis`).

15.2 Importing direct routes into OSPF

```
import route direct
```

Imports all direct routes into OSPF; advertised as **OSPF external routes** across the whole OSPF network. Two ways a directly connected subnet reaches OSPF: (1) enable OSPF on the interface directly, or (2) import the direct route without enabling OSPF on that interface. On an OSPF network, a route flagged **O_ASE** is an OSPF external route (imported, not natively calculated by SPF).

15.3 Importing static routes into OSPF

```
import route static
```

Static routes are **not automatically detected** by dynamic routing protocols – they must be explicitly imported so that all OSPF-domain devices can learn them. Advertised as OSPF external routes.

15.4 Importing IS-IS routes into OSPF

```
import route isis 1
```

Imports all IS-IS routes (from process 1) into OSPF; advertised as OSPF external routes. This is the mechanism used by border routers that bridge an OSPF domain and an IS-IS domain, as described earlier.